

面向原生 GGML FPGA 推理的可见性感知命令流式传输

方法名称: Visibility-Sensitive Trace Compression (VSTC)

研究动机

核心瓶颈是原生、低 batch FPGA 推理里的控制面不匹配。一个 GGML trace 里可能既有必须让主机看见结果、或者会参与可变状态的调用，也有一段只在设备侧流动、并不需要逐个显式完成的 bulk。把这两类调用都当成同样可见，就会把一种语义上的保守处理变成反复的 XRT 启动开销。这里的主要例子是 llama.cpp 在 U280 上运行 Gemma 3 1B F16，但这个问题对所有只在执行时才知道调用图和缓冲区身份的原生张量运行时都是结构性的。

最近的 FPGA LLM 工作让剩下的控制面空缺更明显了。Big PE [openalex:W4416342314] 和 XtraMAC [arxiv:2605.06052v1] 都是在把计算路径推向更高密度或更好的数据类型利用，所以在低 batch 场景里，主机调度会比以前占更大比例的延迟，而不再只是一个小开销。与此同时，llama.cpp 提供了稳定的原生 GGML 调用流，XRT 提供了捕获它所需的命令和完成接口，U280 级板卡也可以容纳一个小型 resident interpreter，而不必替换已安装的 F16 kernels。这样的条件使得直接测试一种运行时表示的变化变得可行，而不是只能从某个模型专用加速器改造去间接推断它的价值。

前人的工作停在结构性原因上。Big PE [openalex:W4416342314] 改的是计算阵列映射，而这种映射里没有主机读取、别名、状态或消费者可见性的运行时记录。LlamaF [openalex:W4405909140] 用的是模型专用的矩阵分组，并把 attention control 和 KV-cache 工作留在 processing system 上，但它没有从真实 GGML trace 里构造可 patch 的 exact templates，也没有在资格判定后给每个原始调用单独暴露完成。XtraMAC [arxiv:2605.06052v1] 提高的是 MAC 利用率，但一个 MAC 微架构本身并不能决定某个运行时调用是否可以延后主机可见的 materialization，而又不改变原生行为。

如果这个空缺被补上，原生 llama.cpp backend 就能在保留原始 tensor allocation 和 explicit-error contract 的同时，避开 prefill 和 decode 中设备侧部分的逐调用主机启动。这样就得到了一条能在不同 trace 之间复用的运行时路径，而不用把每个新模型都重新改写成一套专门的 matrix schedule；同时，延迟提升也更容易归因于可测量的无谓 submission 减少，而不是归因于一个打包在一起的加速器重设计。

方法

ticketing_and_partition

先记录每个原生调用能暴露什么，再把 trace 分成主机可见的边界和安全的设备侧 bulk。

1. 在 GGML 张量运行时调用和 Xilinx Runtime (XRT) 提交/完成事件外面加一层跟踪代码。给每个原始调用分配稳定的 $call_{id}$ ，并记录算子名、张量句柄、XRT buffer object id、字节偏移和长度、命令队列 id、依赖事件、完成事件、返回码，以及任何显式错误信息。每个调用的票据要检查七类具体事实：主机是否能在这一点读取结果，仍然存活的张量视图是否共享重叠字节，这个调用是否修改共享的运行时或队列状态，在任何主机可见边界之前是否只有一个后续设备端消费者，输出是否仍在调用方原始分配中，临时缓冲区的生命周期或地址是否会被外部共享，以及原始调用是否在这个边界暴露错误。把有序票据轨迹写成 JSON Lines，每个 $call_{id}$ 一条记录，顺序与原生执行一致。原始 GGML 调用 i 的可见性感知票据，记录主机读取、别名、状态、唯一消费者、原始分配、临时空间和显式错误事实。

$$\tau_i = (r_i, a_i, s_i, u_i, o_i, q_i, e_i) \quad (1)$$

为什么：压缩决策必须来自运行中的程序到底能观察到什么，而不能只看 operator 名称，或只看一个静态的模型分组。

2. 对票据轨迹做一次确定性的划分。对每个 $call_{id}$ 输出一条决策记录，包含 $call_{id}$ 、它是主机可见边界 H 还是设备端批量条目 B、被拒绝的具体原因、前驱调用 id 和后继调用 id。只要票据显示主机读取、与另一个存活张量视图有字节重叠、修改共享状态、没有唯一设备端消费者、改变了调用方分配、临时缓冲区会被外部共享，或这个调用边界有显式错误可见，就放入 H。只有这些风险都不存在，并且唯一消费者和原始分配两个事实都为真时，才放入 B。用动态张量使用图和 XRT 事件依赖把连续的 B 调用合成最大连续段，中间由 H 调用隔开，并为每个未准入调用保存机器可读的拒绝原因。准入谓词：只有不存在主机读取、别名、状态、临时空间或显式错误风险，且具有唯一消费者和原始分配的调用才能进入 B 模板。

$$b_i = \mathcal{K}[\neg r_i \wedge \neg a_i \wedge \neg s_i \wedge u_i \wedge o_i \wedge \neg q_i \wedge \neg e_i] \quad (2)$$

为什么：这个 partition 不能跨过定义原生 contract 的 visibility、alias、state、allocation、scratch 或 error 条件。

template_streaming

把可准入的 bulk 变成可精确流式执行的 templates，并在保持边界行为可见的同时运行它们。

3. 对每个连续的 B 段创建一个 B-template 记录，而不是发明新的融合算子。模板头需要包含 $template_{id}$ 、有序 $call_{ids}$ 、已有的 16-bit floating point (F16) kernel 标识、参数槽表、XRT buffer object 绑定、字节偏移、张量形状、步长、元素类型、依赖事件槽、完成槽，以及运行时变化的值对应的补丁位置。字节码主体是一串有序的既有 kernel 调用和 XRT 命令描述符，与原生后端本来会发出的内容一致，只是把会变的部分做成符号槽，并在执行前用当前 GGML 张量绑定填入。每个原始 $call_{id}$ 都保留一个完成槽，这样解释器即使按整个 B 段执行，也能按原生顺序报告每个调用的完成。如果某个绑定不能表示成稳定的 buffer object 加字节偏移和长度，就把这个调用从模板中拒绝出去，并记录 $template_build_denial$ 原因。

为什么：exact templates 保留了已安装的 F16 kernels 和调用方的 allocation 语义，同时去掉了反复在 host 侧重建 device-only submissions 的开销。

4. 用一个常驻解释器执行混合的 H/B 流；解释器维护模板缓存，以及按原始 $call_{id}$ 排序的完成账本。遇到 H 时，同样调用原始 GGML/XRT 路径，在原生后端本来会暴露完成的位置暴露完成，并复制相同的返回码和显式错误内容。跨过这个 H 边界之前，要把按原生顺序必须可见的早期 B 完成状态变为可见。遇到 B 模板时，用当前张量绑定填充符号参数槽，把编码后的命令序列送入常驻解释器，按记录顺序运行已有 kernel，并为每个原始 $call_{id}$ 更新完成账本。除非后续 H 边界要求可见，否则不要把 B 内部的中间完成状态暴露给主机。最终按原生 $call_{id}$ 顺序返回 logits、张量副作用、完成状态和显式错误轨迹；任何解释器内部失败都映射回最早受影响的原始调用。

为什么：这样才能实现声称的 launch reduction，同时又不把 resident engine 变成一个隐藏 native runtime observability 的通用异步调度器。

equivalence_and_measurement

检查流式路径是否仍与原生输出一致，并衡量 *admission* 是否真的带来收益。

5. 对同一批测试用例运行三个匹配版本：原生后端、使用已准入 H/B 流的 VSTC，以及把每个可能的 B 条目都强制退回单独完成但其余仍走相同常驻解释器路径的 VSTC。三者使用相同 prompts、模型权重、张量分配和 XRT 配置。最终 logits 要逐字节比较，显式错误轨迹要按包含 $call_{id}$ 、返回码和错误内容的有序记录比较；一旦不同，报告第一个分歧 $call_{id}$ 。用保存的 H/B 决策计算准入比例 ρ ，也就是基线 XRT 提交中有多少比例被已准入的 B-template 条目代表。同时统计实际 XRT 提交次数，并从带时间戳的运行日志中报告 prefill 和 decode 的中位数 (p50) 与尾部 (p95) 延迟；warmup 运行按测量前固定的规则排除。作为全观察检查，重新运行时把所有主机读取事实强制设为真；这应当取消 B 准入，并复现原生完成和错误轨迹。强制拒绝运行用来判断延迟变化是否跟准入比例相关，而不只是来自常驻解释器路径本身。准入比例 ρ ，表示基线 XRT 提交中由已准入 B 模板条目代表的份额。

$$\rho = \frac{\sum_{i=1}^{N_{\text{XRT}}} b_i}{N_{\text{XRT}}} \quad (3)$$

为什么：full-observation oracle 和 forced B-denial control 用来检验任何加速到底是不是来自 visibility-sensitive admission，而不是来自一个没有被单独测量的 interpreter，或者来自一条改过的 kernel path。